Decision Tree Implementation Part 1

Import Libraries

In [ ]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Load Data

In [ ]:
```
data = pd.read_csv('/content/WA_Fn-UseC_-Telco-Customer-Churn.csv')
```

In [ ]:
```
data.head()
```

Out[37]: Columns 1–8 of 22

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService
0	7590-VHVEG	Female	0	Yes	No	1	No
1	5575-GNVDE	Male	0	No	No	34	Yes
2	3668-QPYBK	Male	0	No	No	2	Yes
3	7795-CFOCW	Male	0	No	No	45	No
4	9237-HQITU	Female	0	No	No	2	Yes

Columns 9–15 of 22 · first column repeated

	MultipleLines	InternetService	OnlineSecurity	...	DeviceProtection	TechSupport	StreamingTV
0	No phone service	DSL	No	...	No	No	No
1	No	DSL	Yes	...	Yes	No	No
2	No	DSL	Yes	...	No	No	No
3	No phone service	DSL	Yes	...	Yes	Yes	No
4	No	Fiber optic	No	...	No	No	No

Columns 16–22 of 22 · first column repeated

	StreamingMovies	Contract	PaperlessBilling	PaymentMethod	MonthlyCharges	TotalCharges	Churn
0	No	Month-to-month	Yes	Electronic check	29.85	29.85	No
1	No	One year	No	Mailed check	56.95	1889.5	No
2	No	Month-to-month	Yes	Mailed check	53.85	108.15	Yes
3	No	One year	No	Bank transfer (automatic)	42.30	1840.75	No
4	No	Month-to-month	Yes	Electronic check	70.70	151.65	Yes

5 rows × 21 columns

In [ ]:
```
data.shape
```

Out[38]:
```
(7043, 21)
```

In [ ]:
```
data.columns
```

Out[39]:
```
Index(['customerID', 'gender', 'SeniorCitizen', 'Partner', 'Dependents',
       'tenure', 'PhoneService', 'MultipleLines', 'InternetService',
       'OnlineSecurity', 'OnlineBackup', 'DeviceProtection', 'TechSupport',
       'StreamingTV', 'StreamingMovies', 'Contract', 'PaperlessBilling',
       'PaymentMethod', 'MonthlyCharges', 'TotalCharges', 'Churn'],
      dtype='object')
```

Data Visualization with Sweetviz

In [ ]:
```
!pip install sweetviz
```

```
Requirement already satisfied: sweetviz in /usr/local/lib/python3.10/dist-
packages (2.3.1)
Requirement already satisfied: pandas≠1.0.0,≠1.0.1,≠1.0.2,⩾0.25.3 in
/usr/local/lib/python3.10/dist-packages (from sweetviz) (2.2.2)
Requirement already satisfied: numpy⩾1.16.0 in
/usr/local/lib/python3.10/dist-packages (from sweetviz) (1.26.4)
Requirement already satisfied: matplotlib⩾3.1.3 in
/usr/local/lib/python3.10/dist-packages (from sweetviz) (3.8.0)
Requirement already satisfied: tqdm⩾4.43.0 in
/usr/local/lib/python3.10/dist-packages (from sweetviz) (4.66.6)
Requirement already satisfied: scipy⩾1.3.2 in
/usr/local/lib/python3.10/dist-packages (from sweetviz) (1.13.1)
Requirement already satisfied: jinja2⩾2.11.1 in
/usr/local/lib/python3.10/dist-packages (from sweetviz) (3.1.4)
Requirement already satisfied: importlib-resources⩾1.2.0 in
/usr/local/lib/python3.10/dist-packages (from sweetviz) (6.4.5)
Requirement already satisfied: MarkupSafe⩾2.0 in
/usr/local/lib/python3.10/dist-packages (from jinja2⩾2.11.1→sweetviz)
(3.0.2)
Requirement already satisfied: contourpy⩾1.0.1 in
/usr/local/lib/python3.10/dist-packages (from matplotlib⩾3.1.3→sweetviz)
(1.3.0)
Requirement already satisfied: cycler⩾0.10 in
/usr/local/lib/python3.10/dist-packages (from matplotlib⩾3.1.3→sweetviz)
(0.12.1)
Requirement already satisfied: fonttools⩾4.22.0 in
/usr/local/lib/python3.10/dist-packages (from matplotlib⩾3.1.3→sweetviz)
(4.54.1)
Requirement already satisfied: kiwisolver⩾1.0.1 in
/usr/local/lib/python3.10/dist-packages (from matplotlib⩾3.1.3→sweetviz)
(1.4.7)
Requirement already satisfied: packaging⩾20.0 in
/usr/local/lib/python3.10/dist-packages (from matplotlib⩾3.1.3→sweetviz)
(24.2)
Requirement already satisfied: pillow⩾6.2.0 in
/usr/local/lib/python3.10/dist-packages (from matplotlib⩾3.1.3→sweetviz)
(11.0.0)
Requirement already satisfied: pyparsing⩾2.3.1 in
/usr/local/lib/python3.10/dist-packages (from matplotlib⩾3.1.3→sweetviz)
(3.2.0)
Requirement already satisfied: python-dateutil⩾2.7 in
/usr/local/lib/python3.10/dist-packages (from matplotlib⩾3.1.3→sweetviz)
(2.8.2)
Requirement already satisfied: pytz⩾2020.1 in
/usr/local/lib/python3.10/dist-packages (from
pandas≠1.0.0,≠1.0.1,≠1.0.2,⩾0.25.3→sweetviz) (2024.2)
Requirement already satisfied: tzdata⩾2022.7 in
/usr/local/lib/python3.10/dist-packages (from
pandas≠1.0.0,≠1.0.1,≠1.0.2,⩾0.25.3→sweetviz) (2024.2)
```

```
Requirement already satisfied: six≥1.5 in /usr/local/lib/python3.10/dist-
packages (from python-dateutil≥2.7→matplotlib≥3.1.3→sweetviz) (1.16.0)
```

In [ ]:
```
import sweetviz as sv

# Generate the EDA report
report = sv.analyze(data)

# Show the report in the notebook
report.show_html("sweetviz_report.html")
```

```
                                             |          | [  0%]   00:00 →
(? left)

Report sweetviz_report.html was generated! NOTEBOOK/COLAB USERS: the web
browser MAY not pop up, regardless, the report IS saved in your
notebook/colab files.
```

In [ ]:
```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customerID        7043 non-null   object
 1   gender            7043 non-null   object
 2   SeniorCitizen     7043 non-null   int64
 3   Partner           7043 non-null   object
 4   Dependents        7043 non-null   object
 5   tenure            7043 non-null   int64
 6   PhoneService      7043 non-null   object
 7   MultipleLines     7043 non-null   object
 8   InternetService   7043 non-null   object
 9   OnlineSecurity    7043 non-null   object
 10  OnlineBackup      7043 non-null   object
 11  DeviceProtection  7043 non-null   object
 12  TechSupport       7043 non-null   object
 13  StreamingTV       7043 non-null   object
 14  StreamingMovies   7043 non-null   object
 15  Contract          7043 non-null   object
 16  PaperlessBilling  7043 non-null   object
 17  PaymentMethod     7043 non-null   object
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   object
 20  Churn             7043 non-null   object
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB
```

Data Transformation

In [ ]:
```
# Dropping unnecessary columns (if applicable)
data.set_index('customerID', inplace=True)

# Converting 'TotalCharges' to numeric
data['TotalCharges'] = pd.to_numeric(data['TotalCharges'],
errors='coerce').fillna(0).astype(int)
```

In [ ]:
```
data.head()
```

Out[44]: Columns 1–8 of 21

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines
customerID							
7590-VHVEG	Female	0	Yes	No	1	No	No phone service
5575-GNVDE	Male	0	No	No	34	Yes	No
3668-QPYBK	Male	0	No	No	2	Yes	No
7795-CFOCW	Male	0	No	No	45	No	No phone service
9237-HQITU	Female	0	No	No	2	Yes	No

Columns 9–15 of 21 · first column repeated

	InternetService	OnlineSecurity	OnlineBackup	DeviceProtection	TechSupport	StreamingTV	StreamingMovies
customerID							
7590-VHVEG	DSL	No	Yes	No	No	No	No
5575-GNVDE	DSL	Yes	No	Yes	No	No	No
3668-QPYBK	DSL	Yes	Yes	No	No	No	No
7795-CFOCW	DSL	Yes	No	Yes	Yes	No	No
9237-HQITU	Fiber optic	No	No	No	No	No	No

Columns 16–21 of 21 · first column repeated

	Contract	PaperlessBilling	PaymentMethod	MonthlyCharges	TotalCharges	Churn
customerID						
7590-VHVEG	Month-to-month	Yes	Electronic check	29.85	29	No
5575-GNVDE	One year	No	Mailed check	56.95	1889	No
3668-QPYBK	Month-to-month	Yes	Mailed check	53.85	108	Yes
7795-CFOCW	One year	No	Bank transfer (automatic)	42.30	1840	No
9237-HQITU	Month-to-month	Yes	Electronic check	70.70	151	Yes

Label Encoding

In [ ]:
```
#transfomartion categorical to numerical
from sklearn.preprocessing import LabelEncoder

encode = data[['gender', 'Partner', 'Dependents',
         'PhoneService', 'MultipleLines', 'InternetService',
         'OnlineSecurity', 'OnlineBackup', 'DeviceProtection',
'TechSupport',
         'StreamingTV', 'StreamingMovies', 'Contract', 'PaperlessBilling',
        'PaymentMethod', 'Churn']]
le = LabelEncoder()

for column in encode:
  data[column] = le.fit_transform(data[column])
```

In [ ]:
```
data.head()
```

Out[46]:

Columns 1–8 of 21

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines
customerID							
7590-VHVEG	0	0	1	0	1	0	1
5575-GNVDE	1	0	0	0	34	1	0
3668-QPYBK	1	0	0	0	2	1	0
7795-CFOCW	1	0	0	0	45	0	1
9237-HQITU	0	0	0	0	2	1	0

Columns 9–15 of 21 · first column repeated

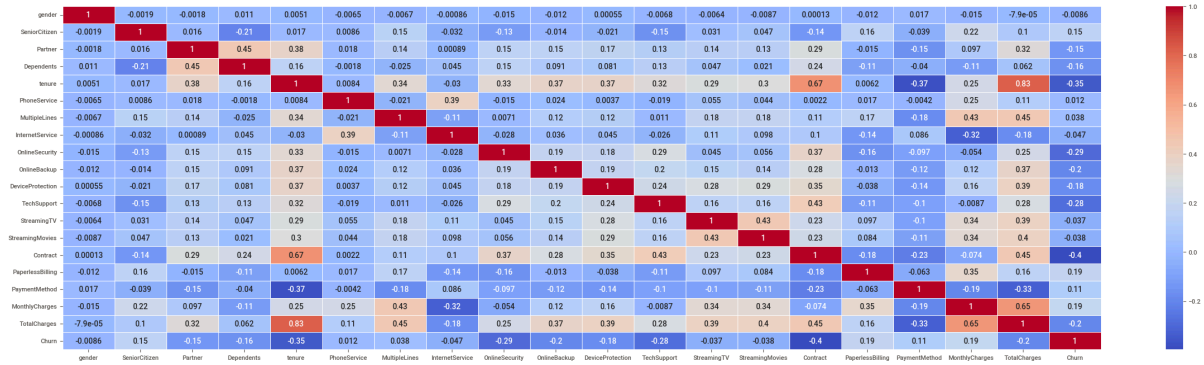
	InternetService	OnlineSecurity	OnlineBackup	DeviceProtection	TechSupport	StreamingTV	StreamingMovies
customerID							
7590-VHVEG	0	0	2	0	0	0	0
5575-GNVDE	0	2	0	2	0	0	0
3668-QPYBK	0	2	2	0	0	0	0
7795-CFOCW	0	2	0	2	2	0	0
9237-HQITU	1	0	0	0	0	0	0

Columns 16–21 of 21 · first column repeated

	Contract	PaperlessBilling	PaymentMethod	MonthlyCharges	TotalCharges	Churn
customerID						
7590-VHVEG	0	1	2	29.85	29	0
5575-GNVDE	1	0	3	56.95	1889	0
3668-QPYBK	0	1	3	53.85	108	1
7795-CFOCW	1	0	0	42.30	1840	0
9237-HQITU	0	1	2	70.70	151	1

Correlation Plot

In [ ]:
```
plt.figure(figsize=(30, 8))
sns.heatmap(data.corr(), annot=True, linewidths=0.5, cmap='coolwarm')
plt.show()
```



As we can seen figure in above, there are five column that have "strong" correlation on the variable object, that is `Churn`

Build DT Model

In [ ]:
```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score, classification_report
from sklearn.metrics import confusion_matrix
```

In [ ]:
```
X =
data[['PhoneService','MultipleLines','PaperlessBilling','PaymentMethod','Mon
thlyCharges']]
y = data['Churn']
```

In [ ]:
```
x_train, x_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

In [ ]:
```
# Train the Decision Tree Classifier
dt = DecisionTreeClassifier(criterion="entropy",random_state=42)
dt.fit(x_train, y_train)
```

Out[51]:
```
DecisionTreeClassifier(criterion='entropy', random_state=42)
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

☑ DecisionTreeClassifier?Documentation for DecisionTreeClassifieriFitted

```
DecisionTreeClassifier(criterion='entropy', random_state=42)
```

DT Model Evaluation

In [ ]:
```
# Predict on training and testing data
train_preds = dt.predict(x_train)
test_preds = dt.predict(x_test)
```

In [ ]:
```
# Calculate accuracy
train_accuracy = accuracy_score(y_train, train_preds)
test_accuracy = accuracy_score(y_test, test_preds)
```

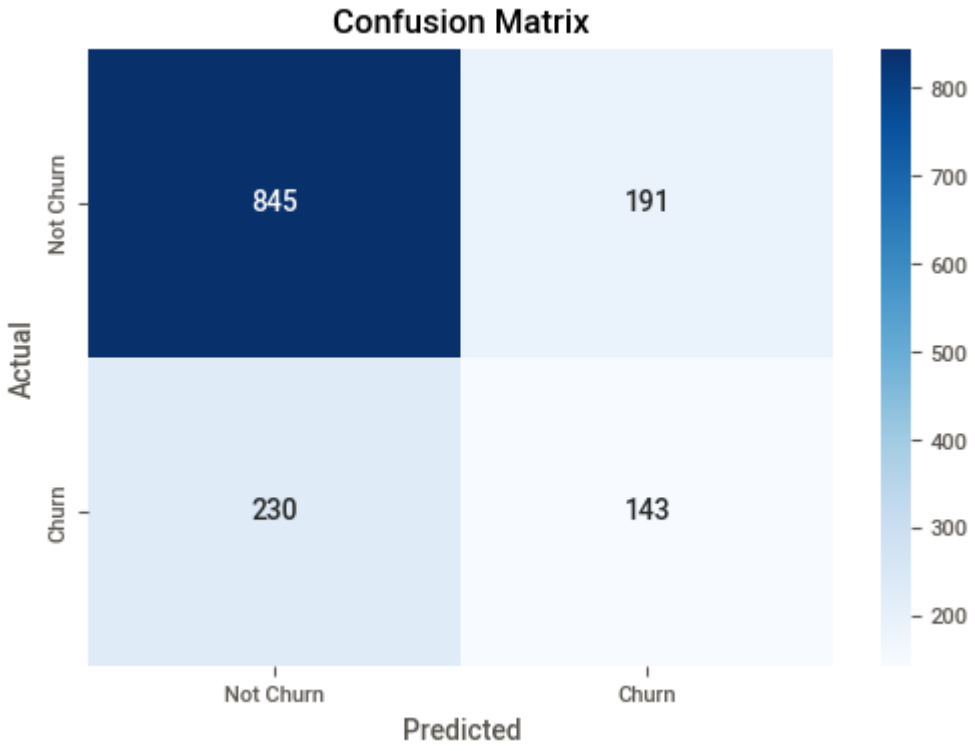
In [ ]:
```
print(f"Training Accuracy: {train_accuracy:.4f}")
print(f"Testing Accuracy: {test_accuracy:.4f}")
```

```
Training Accuracy: 0.9359
Testing Accuracy: 0.7012
```

Confusion Matrix

In [ ]:
```
# Compute confusion matrix
cm = confusion_matrix(y_test, test_preds)

# Plot confusion matrix
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=["Not Churn",
"Churn"], yticklabels=["Not Churn", "Churn"])
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```



- True Negative (`TN`): Correctly predicted negative class (Actual = `Not Churn` , Predicted = `Not Churn`)
- False Positive (`FP`): Incorrectly predicted as positive (Actual = `Not Churn` , Predicted = `Churn`)
- False Negative (`FN`): Incorrectly predicted as negative (Actual = `Churn` , Predicted = `Not Churn`)
- True Positive (`TP`): Correctly predicted positive class (Actual = `Churn` , Predicted = `Churn`)

Classification Report

In [ ]:
```
# classification report
print("Classification Report (Testing Data):\n")
print(classification_report(y_test, test_preds))
```

```
Classification Report (Testing Data):

              precision    recall  f1-score   support

           0       0.79      0.82      0.80      1036
           1       0.43      0.38      0.40       373

    accuracy                           0.70      1409
   macro avg       0.61      0.60      0.60      1409
weighted avg       0.69      0.70      0.70      1409
```

- The model overfits the training data, as seen by the large gap between training (93.59%) and testing accuracy (70.12%).
- The model performs well on predicting Non-Churn (class 0), but struggles with predicting Churn (class 1), as indicated by the low precision and recall for churn.
- The class imbalance (more Non-Churn instances than Churn) is likely contributing to the model's poor performance on the Churn class.
- Consider using techniques like SMOTE (Synthetic Minority Over-sampling Technique) or class-weight adjustments to handle the class imbalance and explore hyperparameter tuning or regularization to reduce overfitting.

Plot Decision Tree

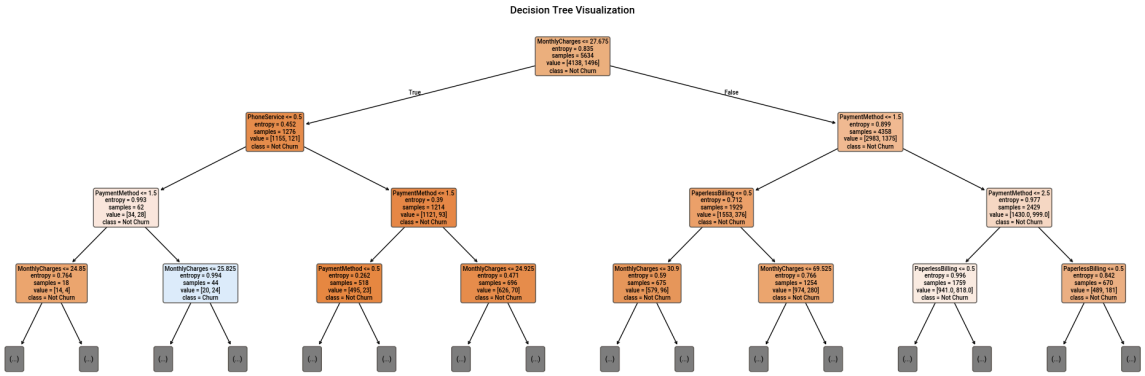
In [ ]:
```
"""from sklearn.tree import plot_tree

# Visualize the decision tree
plt.figure(figsize=(12, 8))
plot_tree(dt, filled=True, feature_names=x_train.columns, class_names=["Not
Churn", "Churn"], rounded=True)
plt.title("Decision Tree Visualization")
plt.show()"""
```

Out[57]:
```
'from sklearn.tree import plot_tree\n\n# Visualize the decision
tree\nplt.figure(figsize=(12, 8))\nplot_tree(dt, filled=True,
feature_names=x_train.columns, class_names=["Not Churn", "Churn"],
rounded=True)\nplt.title("Decision Tree Visualization")\nplt.show()'
```

In [ ]:
```
from sklearn.tree import plot_tree

# Visualize the decision tree
plt.figure(figsize=(25, 8))
plot_tree(dt, filled=True, feature_names=x_train.columns, max_depth=3,
class_names=["Not Churn", "Churn"], rounded=True)
plt.title("Decision Tree Visualization")
plt.show()
```



Decision Tree Implementation Part 2

Cross Validation

In [ ]:
```
from sklearn.model_selection import cross_val_score
```

In [ ]:
```
# k-fold cross-validation
cv_scores = cross_val_score(dt, X, y, cv=5, scoring='accuracy')
```

- `dt` : The Decision Tree model you're using.
- `X` : Your feature matrix (input variables).
- `y` : The target variable (output).
- `cv=5` : This means you're using 5-fold cross-validation (the data will be split into 5 subsets).

- `scoring` ='accuracy': We're interested in accuracy here, but you can choose other metrics such as precision, recall, f1, etc.

In [ ]:
```
print(f"Accuracy for each fold: {cv_scores}")
```

```
Accuracy for each fold: [0.68630234 0.68914123 0.69694819 0.68110795
0.68394886]
```

In [ ]:
```
# Calculate the mean and standard deviation of the accuracy scores
print(f"Mean Accuracy: {np.mean(cv_scores):.4f}")
print(f"Standard Deviation of Accuracy: {np.std(cv_scores):.4f}")
```

```
Mean Accuracy: 0.6875
Standard Deviation of Accuracy: 0.0054
```

Stratified K Fold Cross Validation

In [ ]:
```
from sklearn.model_selection import StratifiedKFold
```

In [ ]:
```
# Initialize StratifiedKFold with k splits
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Perform cross-validation with StratifiedKFold
cv_scores = cross_val_score(dt, X, y, cv=skf, scoring='accuracy')
```

In [ ]:
```
print(f"Accuracy for each fold: {cv_scores}")
```

```
Accuracy for each fold: [0.70120653 0.68062456 0.68559262 0.67045455
0.69176136]
```

In [ ]:
```
# Calculate the mean and standard deviation of the accuracy scores
print(f"Mean Accuracy: {np.mean(cv_scores):.4f}")
print(f"Standard Deviation of Accuracy: {np.std(cv_scores):.4f}")
```

```
Mean Accuracy: 0.6859
Standard Deviation of Accuracy: 0.0103
```

Cross Validation on Other metrics (precision, Recall, F1 score)

In [ ]:
```
# Cross-validation for precision, recall, and f1-score
precision_scores = cross_val_score(dt, X, y, cv=5, scoring='precision')
recall_scores = cross_val_score(dt, X, y, cv=5, scoring='recall')
f1_scores = cross_val_score(dt, X, y, cv=5, scoring='f1')

# Print the results
print(f"Precision for each fold: {precision_scores}")
print(f"Recall for each fold: {recall_scores}")
print(f"F1-score for each fold: {f1_scores}")

# Calculate and print the mean and standard deviation of each metric
print(f"Mean Precision: {np.mean(precision_scores):.4f}, Std:
{np.std(precision_scores):.4f}")
print(f"Mean Recall: {np.mean(recall_scores):.4f}, Std:
{np.std(recall_scores):.4f}")
print(f"Mean F1-score: {np.mean(f1_scores):.4f}, Std:
{np.std(f1_scores):.4f}")
```

```
Precision for each fold: [0.39308176 0.40184049 0.41945289 0.38125
0.3988604 ]
Recall for each fold: [0.3342246  0.35026738 0.36898396 0.32707775
0.37433155]
F1-score for each fold: [0.36127168 0.37428571 0.39260313 0.35209235
0.3862069 ]
Mean Precision: 0.3989, Std: 0.0125
Mean Recall: 0.3510, Std: 0.0186
Mean F1-score: 0.3733, Std: 0.0151
```

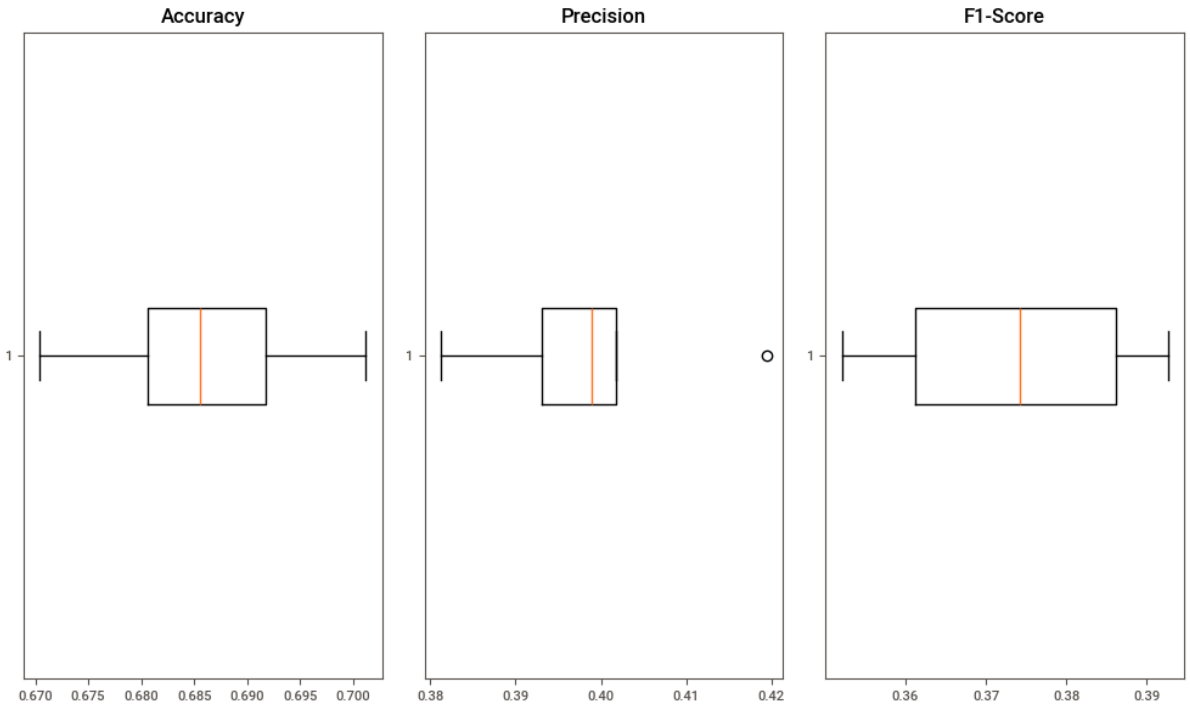
In [ ]:
```
# Plotting the cross-validation scores for each fold
plt.figure(figsize=(10, 6))

# Accuracy plot
plt.subplot(1, 3, 1)
plt.boxplot([cv_scores], vert=False)
plt.title('Accuracy')

# Precision plot
plt.subplot(1, 3, 2)
plt.boxplot([precision_scores], vert=False)
plt.title('Precision')

# F1-Score plot
plt.subplot(1, 3, 3)
plt.boxplot([f1_scores], vert=False)
plt.title('F1-Score')

plt.tight_layout()
plt.show()
```



Summary

Applying cross-validation resulted in lower performance compared to using a single train-test split, there are several potential reasons for this discrepancy.

1. `Check for Overfitting`: Single train-test split may have led to overfitting. Cross-validation gives a more reliable estimate.
2. `Ensure Proper Cross-Validation Setup`: Use StratifiedKFold for imbalanced classes and more folds (e.g., 5 or 10).

3. `Balance Class Distribution`: Address class imbalance using SMOTE, undersampling, or class weighting.
4. `Tune Hyperparameters`: Ensure you're using the best model parameters by performing hyperparameter tuning.
5. `Check Model Complexity`: Avoid overly complex models (e.g., very deep decision trees) that overfit.
6. `Evaluate Multiple Metrics`: Don’t rely on accuracy alone—use metrics like precision, recall, and F1-score.